

실내 순찰 로봇 — 순찰 / 소화기 점검 / 안전모 감지

방 하나를 자율로 순찰하면서 소화기 압력계를 점검하고, 안전모를 쓰지 않은 사람을 만나면 멈춰 경보하고 착용을 확인한 뒤 다시 도는 시스템을 만들었다. 이 보고서는 무엇을 만들었는지와 무엇이 안 됐고 왜 그랬는지를 같은 비중으로 적는다.

로봇	연산	센서	순찰 영역
TurtleBot3 Burger	VM 2코어	LDS-01 라이다	1.15 × 0.95 m
Raspberry Pi 3 / OpenCR	Ubuntu 22.04 / ROS 2 Humble	CSI imx219 / USB 웹캠	웨이포인트 4점

01 / 시스템 구성

무엇이 어디서 도는가

라즈베리파이는 모터/라이다/카메라만 담당하고, 지도/경로계획/영상 인식은 전부 VM 에서 돈다. 영상 인식을 로봇에서 하지 않는 것은 프로젝트 초기에 정한 규칙이다 — Pi 3 로는 추론이 버겁고 무선으로 보내는 편이 낫다고 판단했다.

```
라즈베리파이 bringup(모터/라이다/odom) + webcam_node(USB 웹캠 640x480 3fps)
|
| Wi-Fi (이 프로젝트의 가장 큰 병목)
v
VM  nav2_patrol.launch   지도 + AMCL + 경로계획
    patrol_auto.launch   patrol_node . patrol_scheduler . inspect_node . helmet_node
```

노드는 서로를 직접 부르지 않고 토픽으로만 이야기한다. helmet_node 가 /patrol/hold 를 내면 patrol_node 가 진행 중인 Nav2 목표를 취소해 그 자리에 선다. 계획 단계에서 필요하다고 적어뒀던 cmd_vel_mux 는 결국 쓰지 않았다 — 목표를 취소하면 Nav2 컨트롤러가 스스로 멈추므로 /cmd_vel 을 직접 낼 일이 없었다.

02 / 지도와 순찰 경로

좁은 방이 모든 설계를 정했다



자동 탐사를 돌려보니 frontier 가 0개였다. 미탐사가 남은 게 아니라 물리적으로 좁은 것이었다. 빈 공간 6.56m^2 중 벽에서 30cm 이상 떨어질 수 있는 곳이 1.58m^2 뿐이다.

웨이포인트는 손으로 찍지 않고 지도에서 30cm 여유를 만족하는 최대 사각형을 계산해 넣었다. 손으로 찍었을 때는 위치추정 오차가 섞여 y 가 20cm씩 오르내리고 경로가 지그재그가 됐다 — 그래서 두 번 지웠다.

Nav2 파라미터를 왜 고쳤는가

값	기본	조정	이유
inflation_radius	1.0	0.25	벽 주변 1m 가 고비용 구역이면 30cm 경로가 그 안쪽이라 Nav2 가 계속 벗어나려 한다
xy_goal_tolerance	0.25	0.10	사각형 한 번이 1.15m 인데 25cm 오차는 코너를 뭉갠다
max_vel_x	0.22	0.10	벽 여유가 30cm. 후반에는 안전모 판정을 위해 더 낮췄다
use_sim_time	true	false	기본 파일의 오류. 실기에서 시뮬레이션 시계를 쓰면 아무것도 안 움직인다

합정 — turtlebot3 기본 launch 는 파라미터 버전이 달라 Navigation inactive 가 됐다. humble 용 burger.yaml 을 써야 한다. 그리고 RViz 에서 2D Pose Estimate 를 하지 않으면 AMCL cannot publish a pose 가 무한 반복되고 로봇이 전혀 움직이지 않는다.

순찰을 멈추는 기준은 시간이 아니라 바퀴 수다. 사각형이 작아 한 바퀴가 짧고 매 회차 같은 양을 돌아 결과가 일정하다. 시간 기준이면 코너 중간에 끊겨 애매한 자세로 선다.

순찰 스케줄 — 왜 무작위 시각인가

patrol_scheduler 노드가 다음 순찰을 언제 시작할지 정한다. 좌표는 모르고 시각만 안다 — 웨이포인트/바퀴수는 patrol_node 의 몫이고 스케줄러는 순수하게 "언제"만 담당한다.

순찰 종료 (patrol_node 가 /patrol/status 로 "done" 발행)
-> 최소 rest_min(1분)은 무조건 쉼
-> rest_min ~ cycle_min(기본 2분) 사이 균등난수 시점에 다음 순찰 시작

왜 무작위인가. 순찰이 매년 똑같은 간격으로 돈다면 그 주기를 파악한 사람이 로봇이 없는 시간대를 노려 안전모를 벗거나 금지구역에 들어갈 수 있다. 실제 보안 순찰에서도 예측 가능한 시간표는 감시 효과를 떨어뜨린다는 것이 알려진 원칙이다 — "언제 로봇이 올지 모른다"는 불확실성 자체가 감시 효과를 만든다.

왜 시계(정각) 기준이 아니라 '직전 순찰이 끝난 시각' 기준인가. 처음 계획은 "매 시간 0~50분 사이 랜덤"처럼 시계 경계를 기준으로 뽑는 방식이었다. 그런데 순찰 주기를 5분으로 좁히자 이 방식이 깨졌다 — 순찰 자체가 2~3분 걸리는데 시계 기준으로 뽑으면 순찰이 끝난 직후 다음 시각이 이미 지나가 버려 그 주기를 건너뛰거나 쉬는 시간이 0이 되는 경우가 생겼다. "돌아오면 최소 1분은 반드시 쉼"을 지키려면 기준점이 시계가 아니라 직전 순찰이 끝난 시각이어야 한다. 이렇게 바꾸며 최소 간격(min_gap) 파라미터도 없었다 — 다음 시작 간격이 자동으로 (순찰 소요 시간 + 최소 휴식) 이상이 되므로 별도 규칙이 중복이었다. 이 기준점 덕분에 순찰이 6분 걸려도 쉬는 시간은 그대로 보장되고, 순찰이 안 끝나면 다음 순찰도 잡히지 않아(의도된 동작) 겹쳐 발행될 걱정이 없다.

소화기 점검과 순번을 공유한다. 순찰과 점검은 둘 다 Nav2 에 목표를 보내므로 동시에 돌면 서로 목표를 빼앗는다. 그래서 스케줄러가 한 순번에 하나만 배정한다 — 점검 예약이 들어오면 돌던 순찰을 끝까지 마친 뒤 다음 순번을 점검이 가져가고, 끝나면 다시 순찰로 돌아간다. 이 구조 덕분에 주행 중 끼어들기 장치(cmd_vel 차단 등)가 따로 필요 없어졌다.

03 / 소화기 점검

압력계 바늘을 읽는다

압력계 지름은 2~2.5cm 다. 카메라 높이 20cm, 수직 화각 49° 때문에 70cm 쯤 떨어져야 게이지가 프레임에 들어오는데, 그 거리에서 640×480 이면 게이지가 19px 로 바늘을 볼 수 없다. 1640×1232 로 올리면 약 48px 이 되어 판정이 가능하다.

그런데 고해상도는 무선으로 오지 않는다. 1640×1232 jpeg85 는 프레임당 177KB 이고 UDP 로 100조각 넘게 쪼개진다. best_effort 는 조각 하나만 잃어도 프레임 전체를 버리므로 20초에 0장이 왔다. 그래서 스트리밍을 포기하고 로봇에 로컬 저장한 뒤 tar 스트림으로 가져오는 방식으로 바꿨다.

OK change 0.56 match 0.90 ref1 2%



압력계 정상 판정. 정합 0.90, 변화량 0.57. 도착부터 판정까지 30초.

UNKNOWN match 0.72



판정불가. 자세가 틀어져 게이지가 화면 옆으로 밀렸다.

카메라로 자세를 고친다

자세가 틀어지면 게이지가 화면에서 옆으로 밀린다. 그 어긋난 픽셀을 각도로 환산해 같은 좌표에 그만큼 돌린 yaw 로 목표를 다시 보낸다. 지도/AMCL 이 아니라 카메라로 되먹임을 닫으니 위치추정 오차와 무관하다. 게이지를 아예 못 찾으면 좌우 8° 씩 훑는다.

주행 중에는 카메라를 끈다

카메라 스트림을 켜둔 채 주행하면 무선이 포화돼 /scan//odom 이 밀리고 Nav2 가 모든 목표를 거절한다(실측 ping 1000ms). 그래서 사진을 찍는 그 순간에만 로봇에서 카메라를 띄우고 바로 내린다.

항목	결과
흐름	이동 → 정지 대기 → 촬영 3장 → 판정 → 부저 → 기록 → 시작지점 복귀
도착 ~ 판정	30초
정합 / 변화량	0.90 / 0.57 (정상 판정)
부저	정상 = 1번 음 길게 / 이상 = 2번 음 짧게

04 / 안전모 감지

여기서 대부분의 시간을 썼다

USB 웹캠으로 사람을 찾고 안전모 착용 여부를 판정한다. 사람 검출은 처음부터 잘 됐다 — MobileNet-SSD 로 확신도 0.86~1.00 이 안정적으로 나왔다. 문제는 전부 안전모 판정이었다.

먼저 카메라를 띄우는 것부터 막혔다

- camera_ros 는 USB 웹캠에서 RGB888 미지원 으로 죽는다
- 로봇에 v4l2_camera/usb_cam 이 없고, apt 설치하는 sudo 가 필요하다

로봇에 python3-opencv 가 이미 있어서 스트림 노드를 직접 만들었다. 장치 번호는 콧고 빼면 밀리므로 번호가 아니라 장치 이름으로 찾고, 이름이 같은 장치가 둘 나오면 실제로 프레임이 읽히는 쪽을 쓴다.

사고 — 시험 중 노드를 여러 번 강제 종료하다 웹캠이 잠겼다. 원인은 파이썬이 SIGTERM 에서 finally 를 건너뛰어 cap.release() 가 실행되지 않은 것이다. 장치는 열거되고 v4l2-ctl 단발 캡처는 되는데 OpenCV 스트림만 select() timeout 이 됐고, 케이블 재연결과 authorized 토크로도 안 풀려 로봇을 재부팅해야 했다. SIGTERM 핸들러를 넣어 고쳤고, 더불어 카메라를 부팅당 한 번만 열고 유지하도록 설계를 바꿨다.

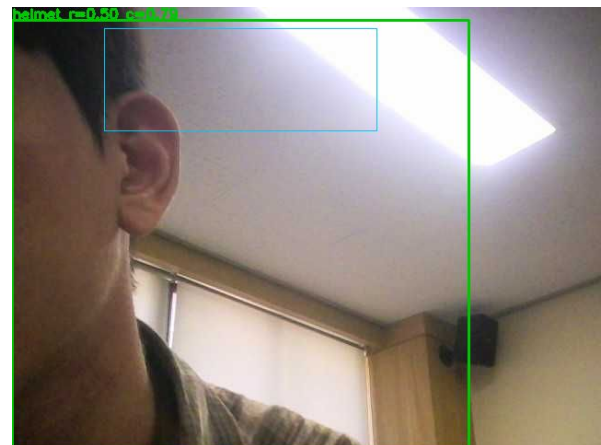
판정 방법을 여덟 번 바꿨다

약 120장의 실제 프레임으로 매번 측정해가며 바꿨다. 아래가 그 기록이다.

#	방법	측정 결과	판정
1	흰 안전모 색(HSV) 매칭	착용 0.037~0.069 / 미착용 0.030~0.048 덩어리 높이가 양쪽 다 144~155px	실패 — 잡힌 게 안전모가 아니라 천장
2	Haar 얼굴 검출 후 그 위를 봄	6장 중 1장만 검출. 맞을 때도 눈 주변 55px (실제 얼굴 200px)	실패 — 환산하면 뺨/이마를 봄
3	사람 상자 맨 위 얇은 띠의 머리카락	착용 0.000(18/18) 미착용 0.064~0.094(15/17)	정지 상태에서에만 성공
4	같은 방식을 순찰 거리에서	0/3 검출	실패 — 상자 top 이 머리보다 20px 위
5	띠를 아래로 오프셋	미착용 39/41 검출 / 착용 오탐 12/18	실패 — 안전모 챙 그림자에 걸림
6	위에서 첫 어두운 덩어리의 높이	착용도 0.16까지 나옴	실패 — 분리 안 됨
7	피부색으로 얼굴을 찾아 그 위를 봄	착용 0~0.535 / 미착용 0~0.687	실패 — 완전히 겹침
8	주변 대비 상대 어두움/두께	착용 0.096~0.287 / 미착용 0.013~0.216	실패 — 착용이 오히려 더 두꺼움



머리카락 방식이 맞은 경우. 띠가 머리카락에 얹혀 $r=0.02$ → 미착용.



같은 방식의 오판. 상자가 화면을 덮자 판정 영역이 흰 천장으로 가서 $r=0.50$ → 착용.



실제 운용 구도. 흰 안전모가 흰 천장/벽과 구분되지 않는다.



순찰 중 오판. 상자 top 이 머리보다 20px 위여서 띠가 벽에 얹혔다 → r=0.000 착용.

왜 전부 막혔나

- 흰 안전모가 흰 벽/천장과 같은 색이다. 그래서 머리가 어디서 시작하는지를 찾을 수 없고, 머리 위치에 의존하는 모든 방법이 무너진다.
- 사람 상자의 위쪽 변이 머리에 딱 맞지 않는다. 프레임마다 0~20px 씩 다르다. 고정된 비율로 띠를 잡으면 어떤 프레임에서는 벽을, 어떤 프레임에서는 천을 본다.

조명도 방해했다. 보정할 때 머리카락은 V 43~100 이었는데, 천장 형광등 때문에 카메라 자동 노출이 화면을 올리자 V 115~157 로 벽(V 152~172)과 같아졌다. 이 웹캠은 노출 제어가 없어(brightness 만 있음) 소프트웨어로 잠글 수도 없었다. 영상 자체에 정보가 없으면 알고리즘으로는 해결되지 않는다.

해결 — 안전모에 색을 넣었다

이 방의 모든 표면은 색상 H 12~22(따뜻한 색) 아니면 무채색이다. 그래서 안전모에 그 대역 밖의 색을 하나 넣으면 판정이 단순해진다. 초록 테이프를 붙였다.

붙이는 위치가 결과를 갈랐다. 처음 안전모 아래쪽 앞면에 붙였을 때는 실패했다 — 바닥 카메라가 올라다보면 그 면이 돔 자신의 그늘에 들어가 H 80~120, S 35~60 으로 밀려 벽/옷과 섞였다. 위쪽 돔으로 옮기자 천장 조명을 받아 H 75~83, S 64~109 로 선명해졌다.



테이프를 아래쪽에 붙인 경우. 그늘에 들어가 색이 죽는다.



위쪽 돔으로 옮긴 최종 형태. 초록 비율 0.023 → 착용.



같은 자리/같은 조명에서 벗은 상태. 초록 비율 0.0000 → 미착용.

같은 자리 / 같은 조명 / 8장씩

초록 비율

착용

0.0199 ~ 0.0289

미착용

0.0000 (8장 전부)

기준선

0.010 — 양쪽 모두 2배 이상 여유

판정 흐름도 바꿨다

처음에는 미착용을 발견하면 바로 세웠다. 그런데 움직이는 동안의 판정이 흔들린다 — 모션블러, 거리 변화, 상자 어긋남. 반면 사람 검출은 안정적이다. 그래서 믿을 수 있는 신호로 먼저 세우고, 멈춘 상태에서 판단하는 순서로 뒤집었다.

사람 2프레임 연속 → 순찰 정지 (안전모 여부와 무관)

5초 동안 판정을 모은다 → 미착용이 사람 프레임의 50% 이상이면 미착용 확정

미착용 → 부저 3연타를 5초마다 반복, 계속 정지, 창을 다시 돌린다

착용 → 부저 1회, 순찰 재개

실주행에서 한 번 더 고쳤다. 미착용 확정 조건이 '몇 장 이상'이라 안전모를 쓰고 있는데도 14장 중 4장이 미착용으로 나와(머리가 화면에서 잘린 프레임) 잘못 세웠다. 진짜 미착용은 14/14/16/16/11/11 로 100% 였으니 개수가 아니라 비율로 바뀌서 같았다.

05 / 두 카메라를 어떻게 나눠 쓰나

항상 한쪽만 켜다

순찰은 USB 웹캠(사람/안전모), 점검은 CSI(압력계)를 쓴다. 둘을 동시에 켜지 않는 이유는 성능이 아니라 무선이다 — 스트림을 하나 더 얹으면 /scan 이 TF 캐시를 벗어나 통째로 버려지고 Nav2 가 경로를 못 만든다.

순찰 차례 webcam 발행 ON - helmet_node 판정
점검 차례 webcam 발행 OFF - CSI 를 그 순간만 띄워 촬영 - 끝나면 되돌림

전환은 helmet_node 가 /inspect/start 와 /inspect/status 의 done 을 듣고 /webcam/enable 로 알린다. 순번 자체는 patrol_scheduler 가 이미 하나씩 배정한다(02절).

USB 를 켜면 CSI 인덱스가 0 → 1 로 밀린다. 안전모 때문에 웹캠이 상시 켜혀 있으므로 점검용 기본값을 1 로 바꿨다. 0 이면 웹캠을 잡아 카메라 노드가 죽는다.

06 / 자원 문제

2코어 VM 과 Wi-Fi

영상 인식을 VM 에서 하기로 한 대가를 후반에 치렀다. Nav2(코스트맵/컨트롤러)와 추론이 같은 2코어를 나눠 쓴다.

프로세스	CPU	비고
Nav2 (component_container)	27.2%	
데스크톱 앱	25.5%	닫으면 확보
RViz	15.6%	2D Pose Estimate 후 불필요
gnome-shell + Xwayland	16.2%	
helmet_node (추론)	12~14%	

전속 판정으로 돌렸을 때 순찰 한 구간이 16초 → 107초로 늘고 중간에 멈추기도 했다. 판정 빈도를 낮추면 감지가 늦어 제때 서지 못했으므로, 빈도는 그대로 두고 우선순위(nice +10)와 OpenCV 스레드 1개로 Nav2 에 CPU 를 양보하게 했다.

무선 쪽은 웹캠이 2.8fps / 34KB/s 로 가벼웠다 — 켜기 전후 /scan 이 4.98Hz 그대로였다. 다만 DDS 디스커버리가 16~40초 걸려서 순찰이 시작될 때 아직 영상이 없는 문제가 있었다. 그래서 시작지점에 도착한 뒤 영상을 기다리도록 했다 — 중간 지점에 멈춰 있으면 고장난 것처럼 보인다는 지적을 받아 그 위치로 옮겼다.

한 회차의 전체 흐름

```

스케줄러가 순번을 준다 (02절 - 직전 순찰 종료 시각 + 무작위 대기)
|
+- 순찰 차례
|   시작지점으로 이동 -> 도착 -> 카메라 영상 확인(최대 40초) -> 출발
|   사각형 4점을 돈다 (0.10 m/s, 코너마다 정지/회전)
|       사람 발견 -> 즉시 정지 -> 5초 판정
|           미착용 -> 부저 3연타 / 5초마다 반복 / 계속 정지
|           착용   -> 부저 1회 / 그 지점부터 이어서 주행
|   바퀴 수를 채우면 스스로 멈춘다
|
+- 점검 차례
    webcam 발행 OFF -> 소화기 이동 -> CSI 촬영 -> 압력계 판정
    -> 부저 / 기록 -> 시작지점 복귀 -> webcam 발행 ON
  
```

검증 항목	결과
Nav2 좌표 이동 / 사각형 순찰	동작
순찰 스케줄(무작위 재시작/순번교대)	시뮬레이션/실주행 동작
소화기 압력계 판정 (실주행)	정상 판정 / 정합 0.90 / 30초
사람 검출	확신도 0.86~1.00
안전모 판정 (초록 테이프)	착용 0.0199~0.0289 / 미착용 0.0000
정지 → 판정 → 재개 (실주행)	사이클 반복 동작
부저 구분	미착용 3번 3연타 / 착용 1번
웹캠이 /scan 에 주는 영향	없음 (4.98 → 4.98Hz)

알고 있는 한계

- 안전모에 색 표시가 필요하다. 흰 안전모만으로는 이 환경에서 판정할 수 없다는 것을 8가지 방법으로 확인했다. 색이 있는 안전모를 쓰면 표시 없이 된다.
- 무선이 병목이다. ping 이 5~1500ms 사이를 오간다. 로봇 신호는 -29dBm 으로 강하고 VM→공유기가 200ms 이므로 호스트 PC 를 유선 랜에 꽂는 것이 근본 해결책이다.
- VM 이 2코어다. 4코어로 늘리면 주행과 추론이 다투지 않는다.
- 배터리. 3셀 리튬폴리머가 11V 아래로 떨어지면 OpenCR 이 경고음을 내고 turtlebot3_node 가 죽는다. 주행 시험은 SMPS 어댑터가 낫다.
- 미검증 — 순찰과 점검의 실제 순번 교대(코드/신호는 검증됨), 다른 사람/다른 조명, QR 내용연한.

측정값은 모두 이 프로젝트의 실제 로그에서 가져왔다. 사진은 logs/inspect_* / logs/helmet_* / logs/helmet_calib/ 의 원본이다. 판정 프레임별 수치는 logs/helmet_<시각>.csv 예, 점검 결과는 logs/inspect_<시각>.csv 에 남아 있다.